Project Overview

Project Title

BreakCheck: API Change Impact Mapper
"If I ship this API version, who breaks?"

Team Almost Avengers
Harsheel Kumar Sahoo, Dhruv Papaiya, Tanishka Jethani

NEXUS UNCHARTED: MongoDB Student Hackathon
Track 3: Developer Tools for Production Use Cases

Problem

When a team changes an API, nobody knows who will break until something fails in production. Teams find out from failed builds, angry users, or late-night incidents. Standard diff tools show which text changed, but not which consumers are affected, so the fix list stays vague: "something broke, someone look into it." Breaking changes slip into releases because the link between an API change and the services that depend on it lives in people's heads.

Solution Summary

BreakCheck answers one precise question: **"If I ship this API version, who breaks?"**

A developer provides two versions of an OpenAPI 3.x spec and a small consumer map (which services call which endpoints and fields). BreakCheck detects every change, classifies it as breaking, potentially breaking, or safe, and matches those changes against what each consumer actually uses. A consumer that never reads a removed field is not flagged. The result is a release verdict (Safe to ship, Ship with caution, or Do not ship) and a migration task list per team, each with a fix hint. All logic is rule-based and deterministic: the same input always gives the same answer, with no LLM involved. Every version, change, run, and event is stored in MongoDB, so history can be compared over time.

Main Features

- **Spec ingestion and validation:** upload YAML or JSON OpenAPI specs. Invalid specs are rejected with the exact location of each problem, and duplicate uploads are detected by content checksum.
- **Deterministic diff engine:** one declarative, unit-tested rule table classifies changes by direction. For example, adding an enum value is safe in a request but potentially breaking in a response.
- **Precise impact analysis:** a MongoDB $lookup aggregation joins changes to consumers by method, path, and the exact field each consumer uses, shown as an impact matrix.
- **Migration tasks:** template-based tasks per consumer with fix hints (such as a likely replacement field). Task status is saved and survives re-runs.

- **Release verdict:** Safe to ship, Ship with caution, or Do not ship.
- **CLI for CI:** runs without a database and exits 0 (safe), 1 (breaking changes hit consumers), or 2 (bad input), so a breaking release can fail a pipeline.
- **Live activity feed:** a MongoDB change stream pushed over Server-Sent Events updates the feed without refreshing, with a polling fallback on serverless hosts.
- **Idempotent webhook:** POST /api/webhooks/spec ignores retries of the same eventId using a unique index.
- **Insights:** consumer risk ranking and breaking-changes-per-release trend chart.
- **Reports and usability:** Markdown report export, a PR-comment summary, and dark mode.
- **Failure handling:** invalid specs, duplicate uploads and webhooks, incomplete consumer maps (warnings, not crashes), and failed analysis runs stored with their error.

MongoDB Usage

Seven collections (projects, specVersions, changes, consumers, analysisRuns, migrationTasks, events) with $jsonSchema validation on required fields while raw specs and payloads stay flexible. Unique indexes and stable hash _ids make ingestion repeat-safe. Eight query-driven indexes are verified with explain() (all IXSCAN). Four aggregation pipelines power impact, summary, risk ranking, and trend.

Verified Results

- v1 → v2: 5 breaking, 3 potential, 11 safe changes; 8 migration tasks; verdict **Do not ship**
- v2 → v3: 7 breaking, 1 potential; 9 migration tasks
- 53 unit tests passing (55 after the report tests are added)

Links

- Live app: *https://breakcheck-live.vercel.app/*
- GitHub: *https://github.com/Harsheel-Kumar-Sahoo/BreakCheck*